

バックプロパゲーション

勾配を全パラメータに届ける仕組み

rkskmt

1

前回の振り返り

- 変換を重ねると **曲がる境界** が作れる（ReLU の折り目）
- 学習に必要なのは、**全パラメータの勾配 $\partial L / \partial \theta$**
- 前回は「誤差が逆向きに流れる」とだけ言って、種明かしを保留した

📌 今回の問い

パラメータが5個、100個、100万個 — **全部の勾配**をどうやって計算する？

2

正面から微分すると地獄

- 前回の2層ネット。 α_1 の勾配を **直接** 書き下すと：

$$\frac{\partial L}{\partial \alpha_1} = \frac{\partial}{\partial \alpha_1} \left[-\log \sigma \left(w \cdot \text{ReLU}(\alpha_1 x_1 + \alpha_2 x_2 + b) + c \right) \right]$$

📌 クイズ

パラメータは5個。この調子で5本の式を全部展開する？ 100層・100万パラメータになったら？

- 実は、**展開しなくていい**
- 鍵は高校数学で習った **合成関数の微分（連鎖律）**

3

連鎖律：微分は掛け算でつなげる

- 関数が数珠つなぎなら、微分も数珠つなぎ：

$$y = f(g(x)) \Rightarrow \frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

📢 クイズ（高校の道具の再確認）

$y = (3x + 1)^2$ を、**展開せずに**微分すると？

- 外側 u^2 の微分は $2u$ 、内側 $3x + 1$ の微分は 3 。掛けて
$$\frac{dy}{dx} = 2(3x + 1) \times 3 = 18x + 6$$
- 展開して微分 ($9x^2 + 6x + 1 \rightarrow 18x + 6$) と一致 — **部品の微分を掛けるだけで、展開は要らなかった**
- 今日はこの1つの道具だけで、NN の全パラメータの勾配を出す

NN は数珠つなぎそのもの

- 「アフィン → ReLU → アフィン → シグモイド → 損失」 — NN は関数の **数珠つなぎ**

$$\frac{\partial L}{\partial \alpha_1} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial u} \cdot \frac{\partial u}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial \alpha_1}$$

- 各カケラは **どれも1行で書ける簡単な微分**
- カケラを **出力側から順に** 掛けて貯めていけば、途中の計算を **使い回せる**

今日のネットワーク（前回と同じ5パラメータ）

$$x \xrightarrow{z=\alpha_1x_1+\alpha_2x_2+b} z \xrightarrow{\text{ReLU}} h \xrightarrow{u=wh+c} u \xrightarrow{\sigma} p \xrightarrow{-\log} L$$

これを **具体的な数値で1往復** する：

記号	値
入力 x_1, x_2	1.0, 0.5
正解 y	1 (Salmon)
α_1, α_2, b	0.3, −0.2, 0.1
w, c	0.5, −0.1

往路 (Forward) : 数値を流す

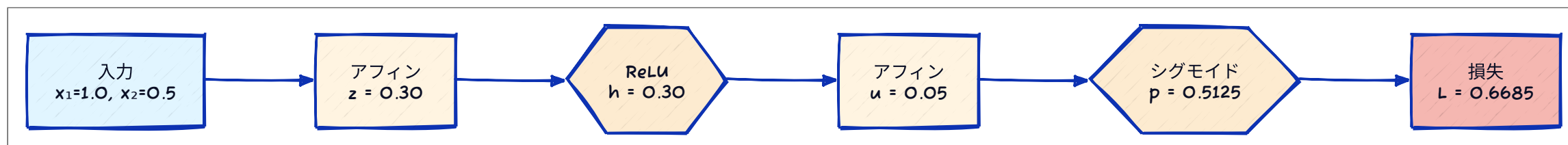
$$z = 0.3 \times 1.0 + (-0.2) \times 0.5 + 0.1 = 0.30$$

$$h = \text{ReLU}(0.30) = 0.30$$

$$u = 0.5 \times 0.30 + (-0.1) = 0.05$$

$$p = \sigma(0.05) = 0.5125$$

$$L = -\log(0.5125) = 0.6685$$



- 予測は 0.5125 — 「ほぼコイン投げ」。正解は 1 なので、もっと上げたい
- **途中の値 (z, h, u, p) は捨てずに取っておく** — 復路で全部使う

7

復路 (Backward) : 出力側から勾配を貯める

出力層

$$\delta_{\text{out}} = \frac{\partial L}{\partial u} = p - y = 0.5125 - 1 = -0.4875$$

(シグモイド+交差エントロピーの「打ち消し合い」 — 最小単位の回の道具)

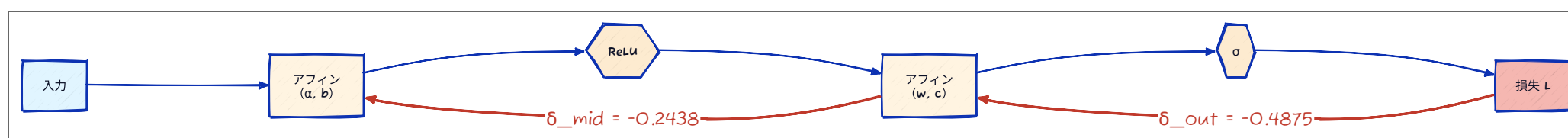
$$\frac{\partial L}{\partial w} = \delta_{\text{out}} \cdot h = -0.4875 \times 0.30 = -0.1463 \quad \frac{\partial L}{\partial c} = \delta_{\text{out}} = -0.4875$$

中間層へ「逆流」

$$\delta_{\text{mid}} = \delta_{\text{out}} \cdot w \cdot \text{ReLU}'(z) = -0.4875 \times 0.5 \times 1 = -0.2438$$

$$\frac{\partial L}{\partial \alpha_1} = \delta_{\text{mid}} \cdot x_1 = -0.2438 \quad \frac{\partial L}{\partial \alpha_2} = \delta_{\text{mid}} \cdot x_2 = -0.1219 \quad \frac{\partial L}{\partial b} = \delta_{\text{mid}} = -0.2438$$

- 5個の勾配が**掛け算と代入だけ**で出揃った。新しい微分は1つもしていない



- 黒い矢印が**往路**、赤い矢印が**復路** — 誤差 δ が右から左へ流れ、通りかかった層のパラメータに勾配を置いていく

8

検算：機械に「微分の定義」でやらせる

- 微分の定義 $\frac{L(\theta + \varepsilon) - L(\theta)}{\varepsilon}$ で、手計算を検算できる（**数値微分**）

Code

```
1 import numpy as np
2
3 x1, x2, y = 1.0, 0.5, 1.0
4
5 def loss(a1, a2, b, w, c):
6     z = a1*x1 + a2*x2 + b
7     h = max(0.0, z)
8     u = w*h + c
9     p = 1 / (1 + np.exp(-u))
10    return -np.log(p)
11
12 params = dict(a1=0.3, a2=-0.2, b=0.1, w=0.5, c=-0.1)
13 base = loss(**params)
14 print(f"L = {base:.4f}")
15
16 eps = 1e-6
17 for name in params:
18     shifted = dict(params)
19     shifted[name] += eps
20     grad = (loss(**shifted) - base) / eps
21     print(f"∂L/∂{name} = {grad:.4f}")
```

Result

```
L = 0.6685
∂L/∂a1 = -0.2438
∂L/∂a2 = -0.1219
∂L/∂b = -0.2438
∂L/∂w = -0.1463
∂L/∂c = -0.4875
```

- 手計算と **完全一致** (a1: -0.2438, a2: -0.1219, b: -0.2438, w: -0.1463, c: -0.4875)
- ではなぜ実用で数値微分を使わない？ — パラメータ100万個なら loss を **100万回** 計算することになる。backprop は **1往復** で全部出る

9

一般形：何層でも同じ漸化式

- 層がいくつあっても、復路は**同じ形の繰り返し**：

$$\delta^{(\ell)} = \delta^{(\ell+1)} \cdot w^{(\ell+1)} \cdot f'(z^{(\ell)})$$

- 「次の層の δ に、重みと活性化の微分を掛けるだけ」
- 各層のパラメータの勾配は $\delta^{(\ell)} \times$ （その層への入力）

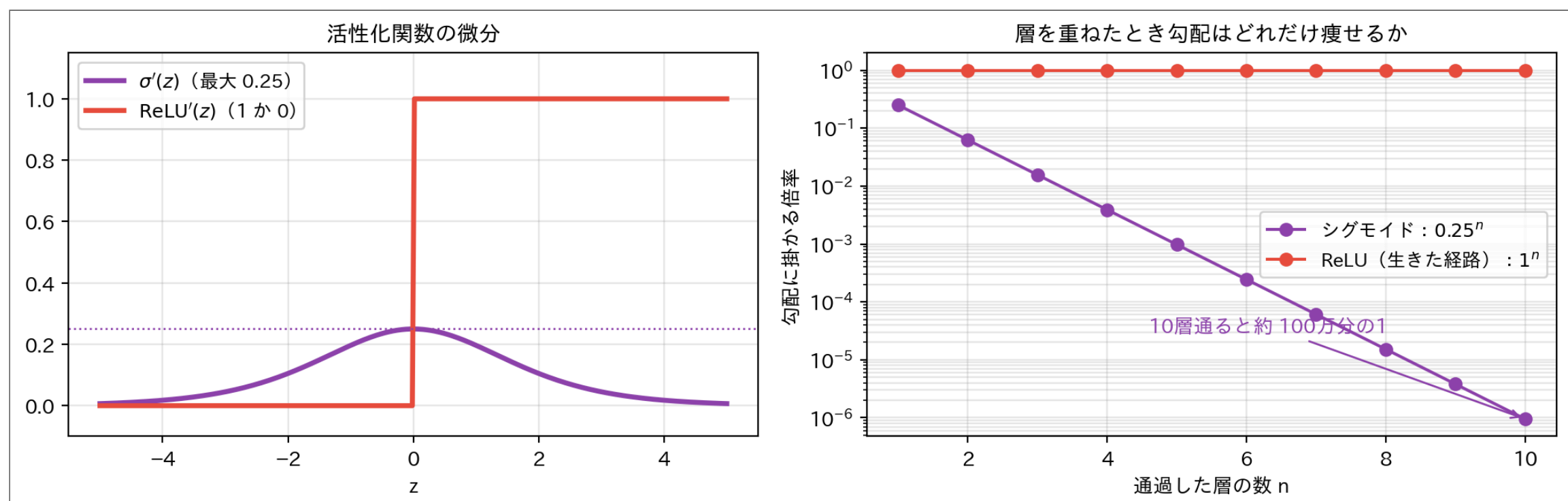
① 名前の由来

誤差（ δ ）が出力から入力へ**逆向きに伝播**していく — **誤差逆伝播法（backpropagation）**。仕組みの正体は連鎖律の使い回し。

10

復路で効いてくる設計たち

- 往路の途中の値 (z, h) を保存しておくのは復路で使うため
 - ディープラーニングがメモリを食う理由のひとつ
- $\text{ReLU}'(z)$ は **1 か 0** — 掛けても勾配が痩せない（生きている経路はそのまま通す）
 - シグモイドを中間に使うと微分が最大 0.25 — 層を重ねるたびに勾配が薄まる（**勾配消失 (vanishing gradient)**）
 - 中間層に ReLU が選ばれる大きな理由



💡 ヒント

最小単位の回の「 $p - y$ のきれいさ」も、ReLU の「1 か 0」も、ぜんぶ**復路の掛け算を壊さないための部品選び**だった。

11

まとめ

- 勾配の計算は、**連鎖律で出力側から掛けて貯める** — 展開不要、微分は部品ごと
- 手計算1往復：forward の値を保存 → δ を逆流させて各勾配
- **数値微分**は検算には最高、実用には遅すぎる
- 何層でも同じ漸化式 $\delta^{(\ell)} = \delta^{(\ell+1)} w^{(\ell+1)} f'(z^{(\ell)})$ が回るだけ

① 次回予告

部品はすべて揃った。次回、**NumPy だけで**多層ネットワークを組み、講義の初回に見せた「機械が自力で見つけた曲がる境界」を**自分のコードで再現**する。

12

練習問題

以下の問題を **手計算** で解き、**数値微分** で検算せよ。
ネットワークは本編と同じ5パラメータ。値だけ差し替える：

記号	値
入力 x_1, x_2	0.5, 1.0
正解 y	0 (Sea bass)
α_1, α_2, b	0.4, 0.3, -0.1
w, c	-0.5, 0.2

問題1：1往復せよ

- 1. 往路： z, h, u, p, L を順に求めよ（正解が $y = 0$ なので $L = -\log(1 - p)$ ）
- 2. 復路： δ_{out} と δ_{mid} 、そして5つの勾配をすべて求めよ
- 3. 本編の数値微分のコードに値を差し替えて、検算せよ

問題2：入力を $x_1 = -1.0, x_2 = 0.5$ に変えて、もう1往復せよ

ヒント： $\sigma(0) = 0.5$ 、 $\sigma(0.2) \approx 0.5498$ 。この2つしか使わない。

💡 ヒント

手計算の答えは捨てないこと。PyTorch入門の練習問題で、この数値がそのまま再登場する。

問題1の解答例

往路

$$z = 0.4 \times 0.5 + 0.3 \times 1.0 - 0.1 = 0.4$$
$$h = \text{ReLU}(0.4) = 0.4, \quad u = -0.5 \times 0.4 + 0.2 = 0$$
$$p = \sigma(0) = 0.5, \quad L = -\log(0.5) = 0.6931$$

ちょうどコイン投げ。正解は 0 なので、 p をもっと下げたい。

復路

$$\delta_{\text{out}} = p - y = 0.5$$
$$\frac{\partial L}{\partial w} = 0.5 \times 0.4 = 0.2, \quad \frac{\partial L}{\partial c} = 0.5$$
$$\delta_{\text{mid}} = 0.5 \times (-0.5) \times 1 = -0.25$$
$$\frac{\partial L}{\partial \alpha_1} = -0.25 \times 0.5 = -0.125, \quad \frac{\partial L}{\partial \alpha_2} = -0.25, \quad \frac{\partial L}{\partial b} = -0.25$$

w が負なので、中間層に流れる誤差は**符号が反転**する $-\delta$ は素通りではなく、通り道の重みに染まる。

数値微分で検算

▶ 解答例を見る

```
Result
|
|  L = 0.6931
|  ∂L/∂a1 = -0.1250
|  ∂L/∂a2 = -0.2500
|  ∂L/∂b = -0.2500
|  ∂L/∂w = 0.2000
|  ∂L/∂c = 0.5000
|
```

問題2の解答例

往路

$$z = 0.4 \times (-1.0) + 0.3 \times 0.5 - 0.1 = -0.35$$

$$h = \text{ReLU}(-0.35) = 0, \quad u = 0.2, \quad p = \sigma(0.2) \approx 0.5498$$

復路

$$\delta_{\text{out}} = 0.5498, \quad \frac{\partial L}{\partial w} = 0.5498 \times h = 0, \quad \frac{\partial L}{\partial c} = 0.5498$$

$$\delta_{\text{mid}} = 0.5498 \times (-0.5) \times \text{ReLU}'(-0.35) = 0$$

$$\Rightarrow \frac{\partial L}{\partial \alpha_1} = \frac{\partial L}{\partial \alpha_2} = \frac{\partial L}{\partial b} = 0$$

❗ この魚は、ネットの左半分にも何も教えない

$z < 0$ で ReLU に折られた瞬間、 $\text{ReLU}'(z) = 0$ が復路の掛け算を **ゼロで潰す**。勾配が届くのは c だけ (w も $h = 0$ に掛けられて消えた)。

前回の「折られた側には誤差が流れない」を、自分の手で確認できた。